

Materia:

Seguridad en el Desarrollo de Aplicaciones

Alumno:

Jose Martín López Gonzalez

Matrícula:

2023371214

Actividad:

OWASP Top 10: Investigacion y analisis de vulnerabilidades

Grupo:

IDGS 17

Tabla comparativa OWASP

OWASP	Descripción	Ejemplo de Ataque	Impacto Potencial	Método de Mitigación
A01	Los usuarios pueden acceder a recursos o funciones fuera de sus permisos previstos.	Modificar el ID en la URL de /api/user/10 a /api/user/11 para ver datos de otro usuario.	Filtración masiva de datos, modificación o borrado no autorizado de registros.	Validar la propiedad del recurso en el backend basándose en la sesión activa del usuario, no solo en parámetros de la URL.
A02	Exposición de datos sensibles debido a la falta o debilidad de mecanismos de cifrado.	Interceptar tráfico HTTP en texto plano o descifrar contraseñas almacenadas con algoritmos obsoletos como MD5.	Robo de credenciales, exposición de datos financieros o datos personales regulados.	Utilizar protocolos seguros (HTTPS), algoritmos robustos de hashing (como bcrypt) y cifrado fuerte para datos en reposo (AES-256).
A03	Datos no confiables enviados a un intérprete como parte de un comando o consulta SQL, NoSQL o SO.	Introducir ' OR '1'='1 en un formulario de login para evadir la autenticación.	Acceso completo a la base de datos, bypass de autenticación, robo, alteración o destrucción de datos.	Usar consultas parametrizadas (Prepared Statements), ORMs seguros y validación/sanitización estricta de entradas.
A04	Defectos inherentes en la arquitectura del software antes de escribir el código (falta de modelado de amenazas).	Permitir preguntas de recuperación de contraseña basadas en datos públicos fácilmente accesibles.	Vulnerabilidades lógicas de negocio imposibles de parchar mediante código simple; compromiso sistémico.	Implementar prácticas de desarrollo seguro desde el diseño (Secure SDLC) y modelado de amenazas temprano.
A05	Endurecimiento (hardening) de seguridad deficiente, habilitación de funciones innecesarias o mensajes de error detallados.	Dejar las credenciales por defecto (admin/admin) de un panel de administración expuesto a internet.	Acceso total al servidor, conocimiento de la estructura interna del código mediante mensajes de error expuestos.	Deshabilitar características innecesarias, cambiar credenciales por defecto y desactivar mensajes de error detallados en producción.

A06	Uso de bibliotecas, frameworks o dependencias de terceros con vulnerabilidades conocidas y públicas.	Explotar un fallo conocido en una versión antigua de Express o de un paquete de npm para ejecutar código remoto.	Ejecución remota de código (RCE), denegación de servicio o control total sobre el servidor de aplicaciones.	Ejecutar auditorías periódicas de dependencias (npm audit), usar herramientas de análisis SCA y mantener los sistemas actualizados.
A07	Debilidades al validar la identidad del usuario, permitiendo ataques automatizados de fuerza bruta.	Realizar ataques de relleno de credenciales (Credential Stuffing) probando miles de contraseñas filtradas de forma automatizada.	Secuestro de cuentas de usuario (Account Takeover), robo de identidad y fraude.	Implementar autenticación multifactor (MFA), políticas de contraseñas robustas y limitación de tasa de peticiones (Rate Limiting).
A08	Código e infraestructura que no protegen contra violaciones de integridad (ej. deserialización insegura o actualizaciones sin verificar).	Modificar una cookie que contiene un objeto serializado para cambiar el rol de usuario a administrador.	Ejecución remota de código, escalada de privilegios o inyección de software malicioso en la cadena de suministro.	Firmar digitalmente los artefactos, verificar la integridad de los datos recibidos mediante hashes robustos y evitar serializaciones inseguras.
A09	Falta de registro de eventos críticos de seguridad o monitoreo ineficaz que impide detectar ataques en curso.	Un atacante realiza un escaneo masivo de vulnerabilidades durante semanas sin que ningún sistema emita alertas.	Incapacidad para detectar brechas a tiempo, lo que prolonga la presencia del atacante en la red de forma persistente.	Registrar eventos clave (logins fallidos, errores de control de acceso) y centralizar los logs con sistemas de alertas automatizadas.
A10	El servidor web es inducido a realizar peticiones HTTP no deseadas hacia un destino arbitrario, a menudo interno.	Proveer una URL interna (http://localhost:8080/admin) a una función del servidor que genera vistas previas de imágenes.	Acceso a servicios de la red interna ocultos tras el firewall, exfiltración de metadatos de la infraestructura en la nube.	Validar y filtrar rigurosamente las URLs ingresadas por el usuario utilizando listas blancas (whitelists) de dominios permitidos.

Casos Reales por vulnerabilidades

A01: Broken Access Control

- **Caso Uber (2019)**
 - **Cómo fue explotada:** Un investigador descubrió que modificando el parámetro de ID de usuario en ciertas llamadas a la API interna, era posible rastrear la ubicación en tiempo real de los conductores y leer los datos de los viajes de cualquier usuario sin autenticación mutua.

A04: Insecure Design

- **Caso Zoom (2020)**
 - **Cómo fue explotada:** Durante el auge de la pandemia, el fenómeno del "*Zoombombing*" ocurrió porque el diseño original del software permitía que cualquier persona ingresara a una reunión con tan solo adivinar o iterar números secuenciales de ID de sala, sin requerir contraseñas o salas de espera de forma predeterminada.

A07: Identification and Authentication Failures

- **Caso GitHub (2018)**
 - **Cómo fue explotada:** Un fallo temporal en el sistema de autenticación de la plataforma provocó el registro erróneo de credenciales de sesión en texto claro dentro de los logs del sistema interno, lo que exponía temporalmente los datos de autenticación a cualquier personal o subproceso con acceso a la monitorización de logs.

Código vulnerable vs seguro para mínimo 3 vulnerabilidades

Ejemplo 1: A03 - SQL Injection

-

// VULNERABLE: Entrada directa sin sanitizar o parametrizar

```
app.post('/login', (req, res) => {  
  const { username, password } = req.body;  
  
  // Si el usuario ingresa en password: ' OR '1'='1  
  const query = `SELECT * FROM usuarios WHERE user = '${username}' AND pass =  
  '${password}'`;
```

```
  db.query(query, (err, result) => {  
    if (result.length > 0) res.send("Autenticado con éxito");  
    else res.send("Credenciales incorrectas");  
  });  
});
```

-

// SEGURO: Uso de Consultas Parametrizadas (Prepared Statements)

```
app.post('/login', (req, res) => {  
  const { username, password } = req.body;  
  
  const query = 'SELECT * FROM usuarios WHERE user = ? AND pass = ?';
```

// Los valores se pasan en un arreglo separado asegurando su correcta parametrización

```
  db.query(query, [username, password], (err, result) => {  
    if (err) return res.status(500).send("Error del servidor");  
    if (result.length > 0) res.send("Autenticado con éxito");  
    else res.send("Credenciales incorrectas");  
  });  
});
```

-

Ejemplo 2: A01 - Broken Access Control (IDOR)

-

// VULNERABLE: No valida la identidad del propietario de la cuenta

```
app.get('/api/account/:id', (req, res) => {  
  const accountId = req.params.id;
```

// Cualquier usuario logueado puede cambiar el ID de la URL y consultar datos
ajenos

```
  db.query('SELECT * FROM cuentas WHERE id = ?', [accountId], (err, result) => {  
    res.json(result);  
  });  
});
```

-

// SEGURO: Verificación estricta basada en el contexto de la sesión del usuario
const verificarToken = require('./middleware/auth'); // Middleware hipotético de JWT

```
app.get('/api/account/:id', verificarToken, (req, res) => {  
  const accountId = req.params.id;
```

const usuarioAutenticadoId = req.user.id; // Extraído de la sesión/token seguro

```
  db.query('SELECT * FROM cuentas WHERE id = ?', [accountId], (err, result) => {  
    if (err || result.length === 0) return res.status(404).send("No encontrado");
```

// Validar si el recurso pertenece realmente al usuario solicitante

```
    if (result[0].usuario_id !== usuarioAutenticadoId) {  
      return res.status(403).send("No tienes permisos para ver este recurso");  
    }  
    res.json(result);
```

```
  });  
});
```

Ejemplo 3: A07 - Identification and Authentication Failures (Falta de Rate Limiting)

-

// VULNERABLE: No hay control del número de peticiones

```
app.post('/api/login', (req, res) => {  
  const { username, password } = req.body;  
  // Lógica de autenticación que puede llamarse millones de veces por minuto por un bot  
  logicaAutenticacion(username, password, res);  
});
```

-

// SEGURO: Implementación de Rate Limiting para mitigar la fuerza bruta

```
const rateLimit = require('express-rate-limit');  
  
const loginLimiter = rateLimit({  
  windowMs: 15 * 60 * 1000, // Ventana de tiempo: 15 minutos  
  max: 5, // Límite de 5 intentos por IP por ventana  
  message: "Demasiados intentos de inicio de sesión. Inténtalo de nuevo en 15 minutos."  
});
```

// Aplicamos el limitador específicamente al endpoint crítico de login

```
app.post('/api/login', loginLimiter, (req, res) => {  
  const { username, password } = req.body;  
  logicaAutenticacion(username, password, res);  
});
```

Ranking Personal de las 5 Vulnerabilidades más Críticas

- **A03: Injection (Inyección)**

Justificación: Ocupa el primer lugar en mi criterio porque su explotación suele otorgar acceso directo, masivo y sin filtros al activo más valioso de la empresa: la base de datos. Además, sigue siendo una de las fallas más recurrentes en el código backend debido a malas prácticas de concatenación de texto básicas.

- **A01: Broken Access Control (Control de Acceso Roto)**

Justificación: Es sumamente crítica porque las herramientas de escaneo automatizado rara vez la detectan por completo, ya que requiere entender las reglas de negocio específicas de la aplicación. Esto significa que los desarrolladores debemos programarla minuciosamente o arriesgarnos a filtraciones masivas mediante simples alteraciones de IDs.

- **A02: Cryptographic Failures (Fallas Criptográficas)**

Justificación: Representa la última línea de defensa de los datos confidenciales. Si un atacante vulnera la red pero los datos están correctamente encriptados y protegidos con algoritmos robustos, el impacto es mínimo; pero si la criptografía falla, toda la información sensible queda expuesta inmediatamente a la luz pública.

- **A06: Vulnerable and Outdated Components (Componentes Vulnerables y Desactualizados)**

Justificación: El desarrollo moderno se basa fuertemente en dependencias de terceros (como los paquetes de npm). Un solo módulo desactualizado o comprometido en la cadena de suministro puede heredar vulnerabilidades críticas de Ejecución Remota de Código (RCE) a todo nuestro sistema sin que hayamos escrito una sola línea de código incorrecta.

- **A07: Identification and Authentication Failures (Fallas de Identificación y Autenticación)**

Justificación: Si los mecanismos para validar quién es el usuario fallan o son susceptibles a ataques automatizados de fuerza bruta, el resto de las defensas perimetrales se vuelven inútiles. El secuestro de cuentas de administración a través de credenciales débiles invalida inmediatamente los controles de seguridad subsiguientes.

Conclusión

El estándar OWASP Top 10 no debe entenderse únicamente como una lista de vulnerabilidades frecuentes en aplicaciones web, sino como una guía esencial para el desarrollo de software seguro y confiable. Actualmente, los desarrolladores no solo tenemos la responsabilidad de crear sistemas funcionales y eficientes, sino también de garantizar que estos sean resistentes ante posibles ataques informáticos. Debido al crecimiento constante de las amenazas digitales, la seguridad dejó de ser un aspecto secundario y pasó a convertirse en un elemento indispensable desde las primeras etapas de diseño y construcción de cualquier sistema. Conocer y analizar las categorías de riesgo planteadas por OWASP permite adoptar una visión preventiva, enfocada en identificar posibles fallos antes de que sean explotados, en lugar de reaccionar únicamente cuando el daño ya ocurrió.

Dentro de mi formación y práctica profesional, este conocimiento representa una base importante para fortalecer la calidad y confiabilidad de los proyectos de software. Aplicar principios de programación defensiva implica utilizar consultas parametrizadas para prevenir inyecciones SQL, validar correctamente la información recibida por el sistema y establecer controles de acceso seguros siguiendo el principio de mínimo privilegio. De igual forma, resulta necesario integrar herramientas automáticas de revisión y auditoría de dependencias para detectar vulnerabilidades en librerías o componentes desactualizados. En conclusión, incorporar las recomendaciones de OWASP en el desarrollo diario no solo mejora la seguridad de las aplicaciones, sino que también protege la información, la estabilidad y los recursos de las organizaciones frente a riesgos que podrían ocasionar pérdidas importantes.